

35.19. Plus lent qu'un pixel par image : la lenteur

Pour aller moins vite que 1 pixel par image, l'avion n'avance qu'une image sur « lenteur » : lenteur 3, c'est 20 pixels par seconde, trois fois moins vite que le 35.16.

1. Ce qu'il faut comprendre

Le problème : au 35.17, `vitesse` compte des **pixels par image**. Le plus petit pas est **1** — 60 pixels par seconde, l'écran traversé en 2,4 secondes. Encore trop rapide pour piloter tranquillement. Et `vitesse = 0.5` ne marche pas : un `uint8_t` ne garde que des **nombre entiers**.

L'idée : garder le pas d'un pixel, mais ne pas avancer à chaque image. L'avion avance **une image sur lenteur**, et attend pendant les autres. `lenteur = 3` : il avance à l'image 3, à l'image 6, à l'image 9... — **20 pixels par seconde**, trois fois moins vite. Le vol reste **fluide** : chaque pas fait toujours un seul pixel.

Ce qui est nouveau ici : un **compteur**, `compte`. À chaque image, `compte++`. Quand il atteint `lenteur`, on le remet à 0 et on laisse les flèches faire **un pas**. Le reste du temps, les flèches ne font rien.

Déroulé avec `lenteur = 3`, flèche droite tenue : image 1 — compte = 1, rien ; image 2 — compte = 2, rien ; image 3 — compte = 3 ≥ 3, donc compte = 0 et **px + 1** ; image 4 — compte = 1, rien... Un pixel toutes les trois images.

Le tableau des lenteurs : `1` → 60 pixels par seconde (le 35.16) ; `2` → 30 ; `3` → **20** (le réglage de cette leçon) ; `4` → 15 ; `6` → 10, très lent. Ici, **plus le nombre est grand, plus l'avion est lent** — c'est le contraire de `vitesse`.

Les bords : le pas est d'un pixel, donc les tests du 35.16 suffisent : `px < 144`, `px > 0`, `py < 128`, `py > 0`.

À toi : essaie `lenteur = 2`, puis `6`, et garde celle qui te va. Le nombre se change **en haut du programme**, à un seul endroit.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de 16 × 16](#)
- `#include <bouton>` → [0.65.5. La fonction `bouton\(\)` — lire la manette](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **493 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          62 o ← la console, à l'allumage — la mise en route de la console
main()             211 o ← démarrage — ta fonction
```

VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	64 o	← CopierTuiles – 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite()	× 4	104 o sur place, dans main()
bouton()	× 4	20 o sur place, dans main() + LireManette (37 o)
image()	× 1	6 o sur place, dans main() + AttendreImage (28 o)
sprite16()	× 1	0 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Plus lent : l'avion n'avance qu'une image sur « lenteur ».

#include "personnages.cpp" // la liste : AVION
#include <sprite16>        // sprite16() : quatre lutins en carré
#include <bouton>           // bouton() : lit un bouton de la manette
```

2. UNE VARIABLE — LENTEUR

```
uint8_t lenteur = 3; // avance une image sur 3 : 1 = 60 px/s, 2 = 30, 3 = 20, 4 = 15, 6 = 10
```

3. UNE VARIABLE — COMPTE

```
uint8_t compte = 0; // les images comptées depuis le dernier pas
```

4. UNE VARIABLE — PX

```
uint8_t px = 72; // la place de l'avion, en pixels : de 0 à 144...
```

5. UNE VARIABLE — PY

```
uint8_t py = 64; // ...et de 0 à 128. Il part du milieu.
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

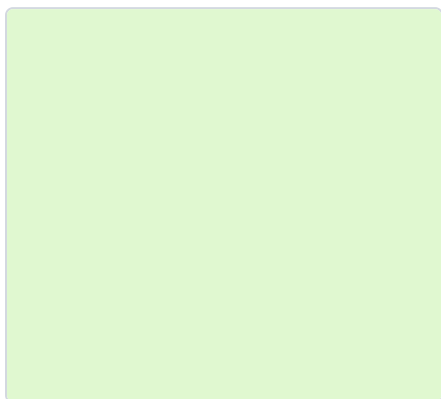
```
int main() {
    while (true) {
        image();

        compte++; // une image de plus
        if (compte >= lenteur) { // c'est l'image d'un pas : une sur « lenteur »
            compte = 0; // on recommence à compter
            if (bouton(DROITE) && px < 144) px++; // un seul pixel : le vol reste fluide
            if (bouton(GAUCHE) && px > 0) px--;
            if (bouton(BAS) && py < 128) py++;
            if (bouton(HAUT) && py > 0) py--;
        }

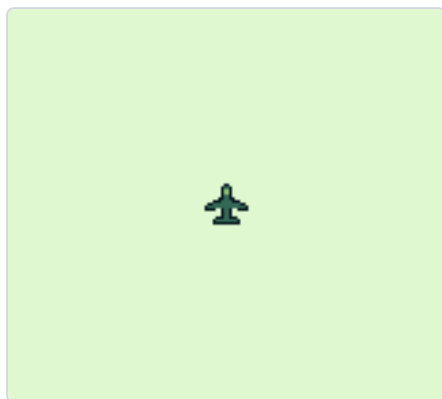
        sprite16(0, px, py, AVION); // posé à CHAQUE image, même sans pas
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



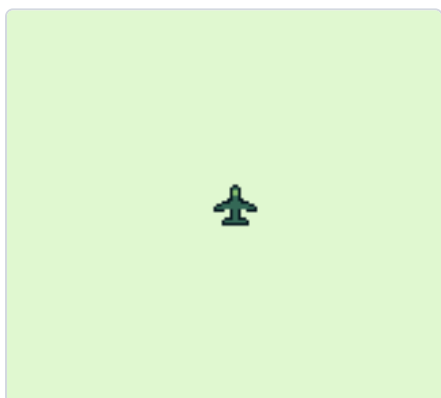
la toute première image — la console vient de démarrer — image 1



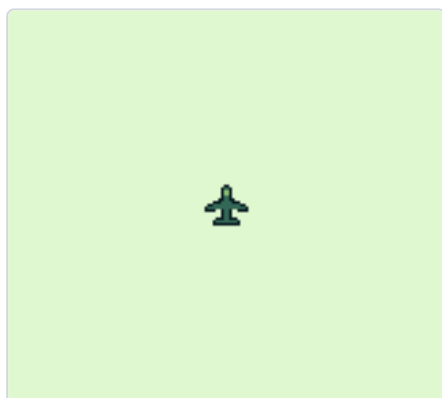
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

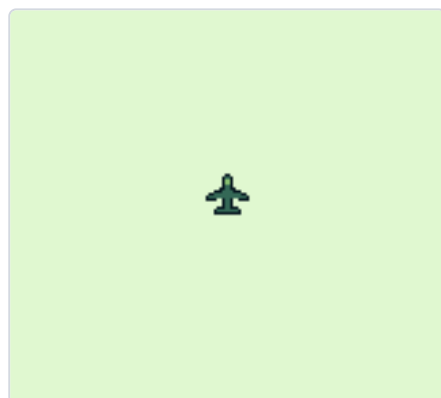
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



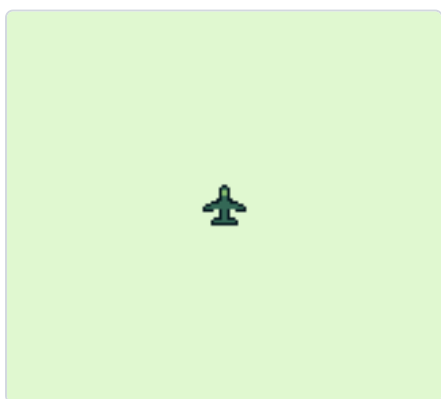
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le même avion, trois fois moins vite qu'au 35.16 : il glisse doucement, un pixel à la fois, et s'arrête toujours au bord.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ 30 images de flèche droite : environ 10 pixels, au lieu de 30 (px = 82)
 - ✓ à gauche, il s'arrête pile à 0
 - ✓ en bas, pile à 128
 - ✓ le lutin suit sa place
-

Ensuite : **36. Intégrer une carte dans son projet** — chapitre 6 : Dessiner. · `gameboy3` — écrire une cartouche Game Boy en C++